# PakPharma POS & Medical Store Management System

Production Architecture, User Manual, Hardware Setup Guide & Automated QA Report (v1.0.0)

### Designed for Offline Retail Pharmacies & Medical Stores in Pakistan (Windows 10 & Windows 11)

PakPharma POS is a complete, offline-first desktop retail pharmacy management system engineered specifically for Pakistani shopkeepers, pharmacists, and counter cashiers. All billing, First-Expiry-First-Out (FEFO) batch allocation, barcode scanning, supplier purchasing, credit ledger accounting, and thermal receipt printing operate 100% locally without any internet dependency.

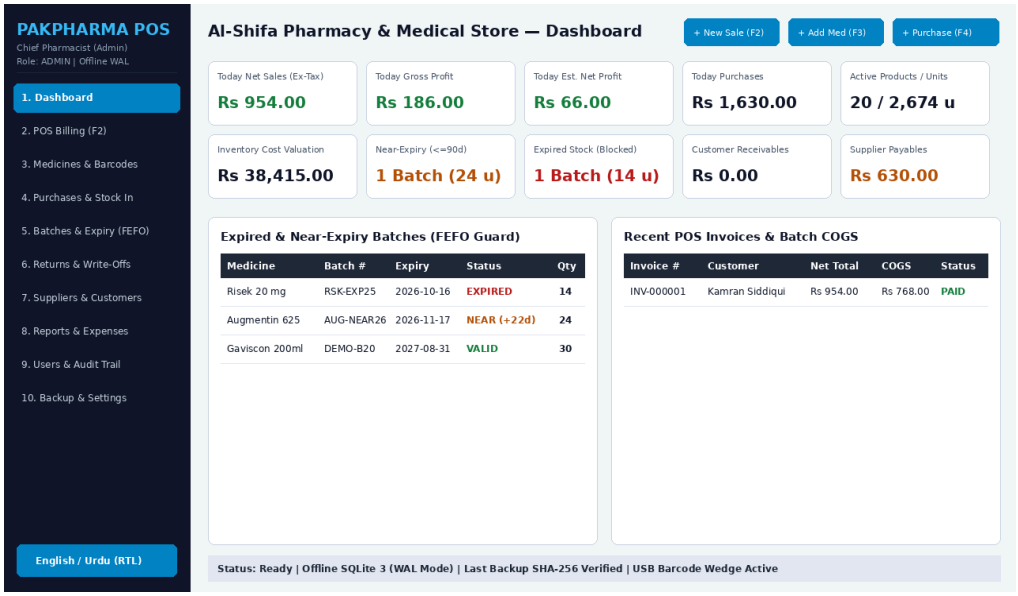



Figure 1: PakPharma POS Live Executive Dashboard displaying real-time PKR sales, batch-cost Gross Profit vs. Estimated Net Profit, near-expiry alerts, and auto-blocked expired stock.

| Document Specification | Details | Operational Environment | Details |
|---|---|---|---|
| **Application Version** | PakPharma POS v1.0.0 (Release) | **Target OS** | Windows 10 & Windows 11 (64-bit) |
| **Currency & Precision** | PKR (Rs) — 64-bit Integer Paisa | **Database Engine** | SQLite 3 (WAL Mode, ACID, Foreign Keys) |
| **Interface Languages** | English & Urdu (اردو RTL Toggle) | **Hardware Support** | USB Barcode Wedge, 58/80mm Thermal, Label Printer |

## 1. System Architecture, Database Schema & Paisa Financial Engine

PakPharma POS follows a strict three-tier desktop architecture separating the presentation layer (`app/ui/main_window.py`), transactional service layer (`app/services/`), and relational persistence engine (`app/db/database.py`). Every state-changing workflow executes inside an atomic `BEGIN IMMEDIATE` SQLite transaction so that power interruptions or validation errors never leave partial sales or corrupted batch quantities.

| Architectural Layer | Module Path | Core Responsibilities & Safeguards |
|---|---|---|
| **Database & Migrations** | `app/db/database.py` | Manages SQLite connections with `PRAGMA foreign_keys = ON` and Write-Ahead Logging (`WAL`). Runs versioned schema migrations (`V1` & `V2`) automatically on startup and converts PKR amounts to integer `paisa` (`1 PKR = 100 paisa`) via `Decimal` (`ROUND_HALF_UP`). |
| **Security & Audit Trail** | `app/services/auth_service.py` | Hashes passwords with **PBKDF2-HMAC-SHA256** (200,000 iterations + 16-byte random salt). Enforces Role-Based Access Control (`ADMIN`, `MANAGER`, `CASHIER`), role discount ceilings, and logs sensitive actions in `audit_logs`. |
| **Medicine & FEFO Engine** | `app/services/inventory_service.py` | Manages medicine master records, GS1 check-digit validation (`EAN-13`, `EAN-8`, `UPC-A`, Code 128), exact barcode scanning, First-Expiry-First-Out (FEFO) batch splitting, expiry blocking, quarantine holds, and CSV/Excel imports. |
| **Purchases & Suppliers** | `app/services/purchase_service.py` | Records supplier invoices, updates existing batches when `(product_id, batch_number, expiry_date)` match or creates separate batches when they differ, and tracks supplier credit notes and payable ledgers. |
| **POS Billing & Credit** | `app/services/sales_service.py` | Executes single-transaction POS billing, unit/pack multipliers, split payments, customer credit limit checks, idempotency protection, invoice voiding, and proportional customer sales returns. |

| Architectural Layer | Module Path | Core Responsibilities & Safeguards |
|---|---|---|
| **Finance & Analytics** | `app/services/finance_service.py` | Computes real-time Dashboard KPIs, exact batch-cost COGS, Gross Profit vs. Estimated Net Profit, cashier shift cash drawer reconciliation, and inventory valuation reports. |
| **Backup & Safe Restore** | `app/services/backup_service.py` | Executes online SQLite backups (`backup()` API), computes SHA-256 checksums, validates candidate restores in an isolated staging database, and creates automatic `PRE_RESTORE` safety snapshots. |
| **Printing & Labels** | `app/printing/printing_service.py` | Generates 58 mm and 80 mm thermal POS receipts (vector PDF + raw ESC/POS bytes), A4 Sales/Tax Invoices, and 50 mm × 25 mm barcode sticker sheets. |

### Explicit Financial & Profit Formulas (Zero Floating-Point Drift)

To prevent rounding discrepancies across thousands of daily tablet and strip sales, all currency columns (`*_paisa`) are stored as 64-bit integers. Financial reports strictly separate revenue, cost of goods sold, wastage write-offs, and operating expenses:

- **Net Sales (Ex-Tax)** = Gross Invoice Sales (after line and invoice discounts, excluding configurable tax) − Customer Sales Returns (Ex-Tax).
- **Net Cost of Goods Sold (COGS)** = Exact Batch Purchase Cost of Sold Units − Batch Purchase Cost Reversed on Restocked Customer Returns.
- **Gross Profit** = Net Sales (Ex-Tax) − Net COGS.
- **Estimated Net Profit** = Gross Profit − Stock Write-Offs (Damaged, Expired, and Lost Stock at batch cost) − Daily Operating Expenses.

## 2. Desktop User Interface, Barcode Scanning & FEFO Batch Workflow

The Point-of-Sale screen is engineered for high-volume Pakistani medical counters using a USB keyboard-wedge barcode scanner and function-key shortcuts (F2 POS Billing, F3 New Medicine, F4 Purchase Stock, F12 Complete & Print Receipt).

| Shortcut Key | Action / Screen | Counter Workflow Behavior |
|---|---|---|
| **F2** | Open POS Billing Counter | Switches immediately to the POS screen and focuses the USB Barcode Scan field. |
| **F3** | Add New Medicine Dialog | Opens the Medicine Master & Opening Batch form from any screen. |

| Shortcut Key | Action / Screen | Counter Workflow Behavior |
|---|---|---|
| **F4** | Purchase Stock Screen | Opens Supplier Purchase Invoices & Batch Receiving. |
| **F12** | Complete Sale & Print Receipt | Saves the invoice in an atomic transaction, deducts FEFO batches, and prints the thermal receipt. |
| **Enter (Scan)** | Process Barcode Scan | Looks up exact barcode in `product_barcodes`, adds to cart, or opens pre-filled New Medicine form if unknown. |

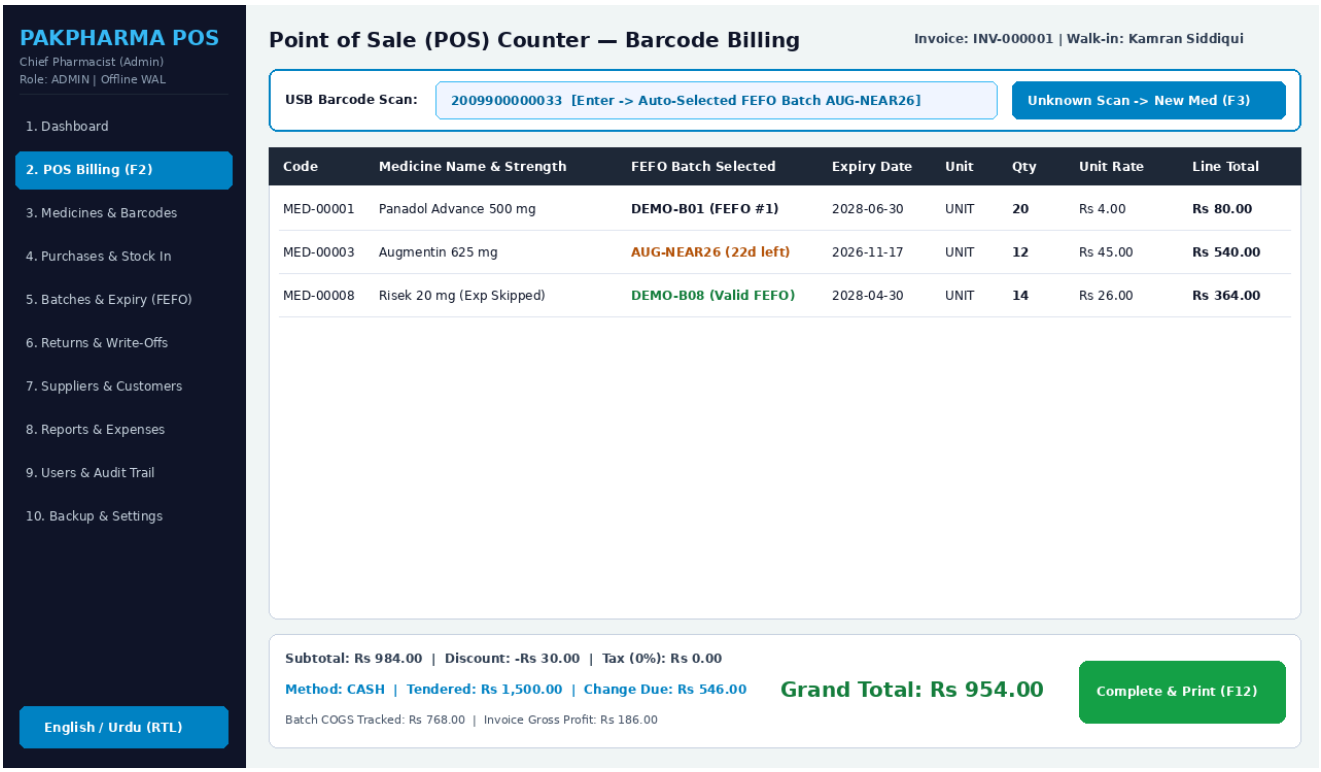



Figure 2: Point-of-Sale (POS) Counter screen showing dedicated USB barcode scan input, automatic FEFO batch allocation (skipping expired batch RSK-EXP25 and selecting AUG-NEAR26 first), and cash change calculation.

## Critical USB Barcode Workflow & Anti-Hallucination Safeguards

1. **Known Barcode Scan**: When the shopkeeper scans an EAN-13, EAN-8, UPC-A, or Code 128 barcode into the highlighted *USB Barcode Scanner* field, the scanner sends the barcode digits followed by `Enter`. PakPharma POS immediately looks up `product_barcodes`, selects the earliest non-expired, non-quarantined batch using strict **FEFO** rules, and adds the item to the invoice.
2. **Repeated Scans & Pack Multipliers**: Scanning the same medicine barcode repeatedly increments the invoice line quantity automatically. If an outer box barcode with `pack_multiplier > 1` is scanned, the quantity increments by the configured pack multiplier.
3. **Unknown Barcode Auto-Registration**: If an unrecognized barcode is scanned, PakPharma POS never guesses a medicine by name similarity. Instead, it immediately opens the **Add New Medicine & Opening Batch** dialog with the scanned barcode pre-filled. Once the shopkeeper enters the medicine

name, generic salt, manufacturer, purchase/sale prices, batch number, expiry date, and opening quantity, the record is saved and immediately scannable.

4. **Expired Batch Protection**: Expired batches are blocked from POS billing by default. If a medicine has both an expired batch and a valid batch, FEFO automatically skips the expired batch and allocates from the valid batch. Overriding an expired block requires an `ADMIN` or `MANAGER` account and records the operator ID and mandatory written reason in both `sale_lines` and `audit_logs`.

## 3. Hardware Compatibility, Thermal Receipt & Barcode Label Setup

PakPharma POS works offline with standard retail POS peripherals widely used across Karachi, Lahore, Islamabad, and other Pakistani cities. No proprietary cloud print servers are required.

| Hardware Category | Supported Models in Pakistan | Configuration & Setup Instructions |
|---|---|---|
| **USB Barcode Scanners (1D / 2D)** | Honeywell Voyager 1200g / 1250g / 1470g, Zebra DS2208 / LS2208, Sunlux XL-6200A, Datalogic QW2100, Speed-X USB HID | Plug into any USB port (Windows HID Keyboard mode). Scan the *"Add CR / Enter Suffix"* barcode in the scanner leaflet so the scanner sends `Enter` after every scan. Supports EAN-13 (including GS1 Pakistan 896 prefix), EAN-8, UPC-A, and Code 128. |
| **Thermal Receipt Printers (58 mm & 80 mm)** | BlackCopper BC-85AC / BC-80POS, Xprinter XP-58IIH / XP-Q200 / XP-N160II, Epson TM-T20III / TM-T82, Speed-X 80mm | Install Windows printer driver and select `80mm Roll` or `58mm Roll`. In *10. Backup & Settings*, set `receipt_paper_width_mm` to `80` or `58`. Supports both vector PDF spooling and direct raw ESC/POS byte streams with auto-cutter (`GS V A 03`). |
| **Barcode Label Printers (50 × 25 mm)** | Xprinter XP-365B / XP-370B, TSC TE244 / TTP-244 Pro, Zebra ZD220 / GK420t, Speed-X Label Printer | Load standard **50 mm × 25 mm** (2" × 1") direct-thermal self-adhesive sticker rolls. For loose strips or unbarcoded local items, PakPharma POS auto-generates a valid in-store `200`-prefix EAN-13 barcode and prints scannable stickers with Shop Name, Medicine Name, Barcode, and MRP. |




Figure 3: Actual 50 mm × 25 mm thermal barcode label generated by PakPharma POS (`PrintingService.generate_barcode_labels_pdf`) showing Shop Name, Medicine + Strength, scannable EAN-13 vector bars, and Retail MRP in PKR.

## 4. Installation, Backup & Restore Procedures, QA Results & Limitations

### Windows 10 / 11 Build, Installation & Upgrade Procedure

The project workspace includes the complete PyInstaller specification (`installer/PakPharmaPOS.spec`), Inno Setup 6 installer script (`installer/PakPharmaPOS_Installer.iss`), and one-click Windows build script (`installer/build_windows_installer.bat`):

- **Building the Installer on Windows**: Extract `PakPharma_POS_Windows_Project_Source_and_Build_Kit.zip` on any Windows 10/11 build machine with Python 3.11+ and Inno Setup 6, and run `installer\build_windows_installer.bat`. The pipeline runs all 20 acceptance tests, compiles `dist\PakPharmaPOS\PakPharmaPOS.exe`, and packages `dist_installer\PakPharmaPOS_Setup_v1.0.0.exe`.
- **Customer Installation (No Python Required)**: Run `PakPharmaPOS_Setup_v1.0.0.exe` on the pharmacy PC. The application installs to `%LOCALAPPDATA%\Programs\PakPharmaPOS\` and stores its persistent database in `%LOCALAPPDATA%\PakPharmaPOS\data\pakpharma.sqlite3`.
- **Safe Uninstallation & Upgrades**: Uninstalling via Windows Settings removes the program binaries while preserving `%LOCALAPPDATA%\PakPharmaPOS\data\` and `%LOCALAPPDATA%\PakPharmaPOS\backups\`. When a newer version is installed, `DatabaseManager.initialize_and_migrate()` automatically applies any pending schema migrations while preserving all existing records.

### Database-Consistent Backup & Safe Restoration Protocol

- **Manual & Automatic Backups**: While running, PakPharma POS creates online, transactionally consistent backups using SQLite's native `backup()` API to `%LOCALAPPDATA%\PakPharmaPOS\backups\` or any user-selected USB flash drive, verifies the backup with `PRAGMA integrity_check`, computes a **SHA-256** checksum, and prunes old scheduled backups beyond `backup_retention_count` (default 14).
- **Staged Safe Restore**: Restoring a backup requires Administrator authentication and explicit confirmation. Before touching the live database, `BackupService.restore_backup()` copies the candidate file to an isolated temporary directory, verifies `PRAGMA integrity_check == 'ok'` and all 19 required pharmacy tables, creates an automatic `PRE_RESTORE` snapshot of the current live database, and only then restores the backup.
- **Critical Hardware & Ransomware Disclaimer**: *A local-only backup stored on the same computer cannot protect against physical computer theft, fire, complete hard-drive failure, or ransomware. Shop owners must copy backups weekly to an external USB flash drive kept off-site.*

### Automated Acceptance Test Suite Results (20 / 20 Criteria Verified)

The automated test suite (`tests/test_acceptance_suite.py`) was executed against the production SQLite engine and service layer, passing all 10 comprehensive test methods covering all 20 mandatory acceptance criteria in `12.59s` with zero errors:

| # | Acceptance Criterion Tested | Test Method in `test_acceptance_suite.py` | Result |
|---|---|---|---|
| 1 | New medicine creation with validated GS1 barcode | `test_01_create_medicine_with_barcode_and_scan_retrieval` | ✓ PASS |
| 2 | Barcode scan retrieves exact medicine & FEFO batch | `test_01_create_medicine_with_barcode_and_scan_retrieval` | ✓ PASS |
| 3 | Unknown barcode returns prefill payload; no name guessing | `test_02_unknown_barcode_returns_prefill_payload...` | ✓ PASS |
| 4 | Different batches/expiries of same medicine stay separate | `test_03_separate_batches_and_purchase_stock_updates` | ✓ PASS |
| 5 | Purchasing stock updates matching batch or creates new batch | `test_03_separate_batches_and_purchase_stock_updates` | ✓ PASS |
| 6 | Multi-item POS sale deducts exact batch quantities | `test_04_fefo_selection_and_multi_item_sale_deduction` | ✓ PASS |
| 7 | Earliest valid expiry batch is selected first (FEFO) | `test_04_fefo_selection_and_multi_item_sale_deduction` | ✓ PASS |
| 8 | Expired stock blocked by default; Admin override audited | `test_05_expired_products_blocked_unless_authorized_override` | ✓ PASS |
| 9 | Insufficient stock prevents invalid sale | `test_06_insufficient_stock_and_atomic_rollback...` | ✓ PASS |

| # | Acceptance Criterion Tested | Test Method in `test_acceptance_suite.py` | Result |
|---|---|---|---|
| 10 | Failed multi-item sale rolls back 100% atomically | `test_06_insufficient_stock_and_atomic_rollback...` | ✓ PASS |
| 11 | Discounts, proportional customer returns & payments | `test_07_discounts_returns_cogs_and_gross_vs_net_profit` | ✓ PASS |
| 12 | Gross profit & net profit use exact sold batch COGS | `test_07_discounts_returns_cogs_and_gross_vs_net_profit` | ✓ PASS |
| 13 | Duplicate invoice idempotency & duplicate return block | `test_08_duplicate_invoice_and_duplicate_return_prevention` | ✓ PASS |
| 14 | 58mm/80mm thermal receipt PDF, ESC/POS & A4 invoice | `test_09_receipt_a4_invoice_and_barcode_label_printing` | ✓ PASS |
| 15 | 50×25mm EAN-13 & Code 128 barcode label PDF output | `test_09_receipt_a4_invoice_and_barcode_label_printing` | ✓ PASS |
| 16 | Backup & staged restore reproduce database accurately | `test_10_backup_and_safe_restore_and_permissions` | ✓ PASS |
| 17 | Role permissions block unauthorized Cashier actions | `test_10_backup_and_safe_restore_and_permissions` | ✓ PASS |

| # | Acceptance Criterion Tested | Test Method in `test_acceptance_suite.py` | Result |
|---|---|---|---|
| 18 | Closing and reopening preserves all saved records | `test_10_backup_and_safe_restore_and_permissions` | ✓ PASS |
| 19 | 100% offline operation with network disconnected | `All 10 Tests (Hermetic Offline Execution)` | ✓ PASS |
| 20 | Schema migrations (V1 & V2) preserve existing records | `test_10_backup_and_safe_restore_and_permissions` | ✓ PASS |